

OmegaHive Deliberation Rooms

Vision, a Slack MVP, and the relation to the Conversation Governor

Cassio Pennachin

drafted with the OmegaHive design-partner session

July 29, 2026

Abstract

OmegaHive's seats increasingly need to deliberate with each other — two projects' design partners arguing a question that spans them — with the human in the loop as a participant rather than a middleman. This note describes what chat rooms look like in a fully functional OmegaHive, specifies a deliberately small MVP on Slack, and maps — honestly, including what the MVP cannot validate — how the hive's design relates to the failure classes documented in the OmegaHive Conversation Governor proposal (v2, July 24, 2026). The claim is narrower than “we don't need a governor”: most of the governor's target failures lose their *generator* in a hive whose agents are never resident and whose reality test is the commit; a few require explicit rules we commit to keeping; and two claims cannot honestly be made until wakes automate. This document is self-contained; architectural background is in the OmegaHive architecture whitepaper (omegahive repository, docs/).

1 The hive in one paragraph

OmegaHive runs work through committed files and an event ledger (the “spine”), not through chat. Work orders are files. *Workers* are task-scoped agent sessions that execute one order and end. *Seats* are durable roles — a memory directory, a protocol, a rehydration bundle — whose *state* persists while their *process* does not: a seat is woken for an occasion, reads its files, acts, and sleeps. Everything durable — decisions, orders, results — lands through a reviewed commit path; today a human operator holds that gate, and the roadmap replaces the human gate with per-seat credentials without changing the path's shape. The consequence that matters here: **the hive has no resident conversational processes**. Nothing sits on a message stream deciding whether to reply.

2 Why rooms

The hive's seats increasingly need to deliberate with each other — two projects' design partners arguing a question that spans them — with the human in the loop as a participant, not as a middleman relaying context between sessions. The operating doctrine already splits this correctly: **chat is where deliberation happens; files are where conclusions land**. Rooms are the chat surface that makes the first half real, in a UI humans already live in, without imposing on the conversation the ceremony that files rightly impose on conclusions.

3 Rooms at full function

Wake-based participation. A room's members are humans and seats. Seats participate the way they do everything: woken for an occasion — a question, an assigned turn, a scheduled deliberation — present for a bounded span, then asleep. The seat's memory remains its files; the room transcript's tail joins its rehydration. Nothing lurks. This is the single load-bearing safeguard in this design, and most of Section 5 rests on it.

A deliberation's lifecycle. Opened with a question, a roster, and a designated *synthesizer*. Conversation follows — see Section 4 for the only rule it has. Terminal state: the synthesizer writes the conclusions into the hive's committed record (decision entries, order drafts, notes), posts the references back into the room, and the transcript is archived under the project that owns the question. A deliberation that ends without committed conclusions is explicitly abandoned — allowed, but said out loud, never silent.

Chairing and synthesizing are different roles. The chair is whoever grants turns and keeps the conversation converging — in the MVP that is naturally the human in the room; once wakes automate, the wake mechanism *is* the chair, because a turn grant is a wake and exclusivity becomes structural rather than social. The synthesizer holds exactly one duty: the terminal artifact. No seat is asked to moderate a conversation it is arguing in.

Identity. Each seat speaks as itself — its own bot identity at the transport level now, ledger-grade credentials once the hive's identity roadmap delivers them. A room's roster is declared; an unattributable voice is a defect.

What the ledger carries. Two events per deliberation: opened (question, roster) and closed (conclusion references) — enough for the audit to answer “which deliberation produced this decision” and for retrospectives to count. Message traffic never touches the ledger.

The metric that guards the doctrine. Retrospectives read the *landing rate*: the fraction of deliberations whose conclusions actually committed. A falling landing rate means rooms are becoming where decisions die — the one failure mode that would make this whole surface a mistake.

The trajectory: chat becomes the operating surface. Deliberation rooms are the first tenant of a larger end state. As per-seat credentials land, more of the operator's day moves into chat: approvals given in a room and executed as signed acts, review conversations, launch requests — the command line receding to plumbing that the chat surface drives. The hive may also grow conversational agents of its own, as ordinary credentialed seats under the same wake discipline. None of this changes the doctrine: every durable act still lands through the write path; the room is where it is discussed and authorized, never where it lives. Separate hive deployments running this same design stay separate — they share the design, not channels.

What rooms are not — ever. Not a write path. Not a task queue (orders and the board remain that). Not a residence (nothing reacts to ambient traffic). Not where knowledge lives (archives are transcript, never truth).

4 The one rule

Nothing said in a room is real until it is committed through the write path. Decisions, conclusions, and requested work exist when they land as committed files — before that they are conversation, however confident their phrasing. A deliberation therefore ends in commits or in explicit abandonment; that is the entire protocol. There are no turn formats, no message contracts, no word budgets: the conversation flows the way conversation flows between people who share a discipline about what counts as done. The hive’s evidence culture — claims about the state of the world carry references into the record — travels into rooms as habit, not enforcement.

5 Relation to the Conversation Governor

The OmegaHive Conversation Governor design (v2, 2026-07-24) addresses real, transcript-documented failure modes of resident chat agents, and its method — shadow-first deployment, predeclared activation gates, cost-bounded governing — is the right way to retrofit discipline onto a live conversational system. The claim of this note is narrower than “we don’t need it”: most of the governor’s target failure classes lose their *generator* in a hive whose agents are never resident and whose reality test is the commit; a few need explicit rules we commit to keeping; and two claims we cannot honestly make yet are marked in Section 7.

OCG failure class	Disposition here	Why
Acknowledgment loops	dissolved	nothing wakes on an acknowledgment; a message triggers no response by default, so an ack cascades to nobody
Visible silence markers (NO_REPLY prose)	dissolved	silence is the default state of a system with no resident responder; there is no invocation to suppress
Parallel public answers	dissolved as accident, retained as choice	nothing auto-answers, so <i>unrequested</i> parallel answers have no generator; in a live deliberation the chair may ask for parallel independent takes — the governor’s /all-answer as a deliberate move; with per-turn wakes, exclusivity is structural
Tool narration in channel	dissolved	execution lives in workers; workers are not in rooms; rooms carry no operational stream
System-event echoing	solved by a kept rule	monitoring/notification streams and deliberation rooms are separate channels, and seats never wake on monitor traffic — a rule, not an accident, since one transport may host both
State reconstruction	solved by existing design	seats rehydrate from compact committed state files; the “compact project-state snapshot” of the governor design (§6.8 there) exists here as each project’s current-state docs, already load-bearing
Channel/identity confusion	residual, transport-dependent	minor on Slack (native per-bot identity); real on Telegram (Section 8); ledger-grade identity arrives with seat credentials
Promise loops	defanged by the one rule	an in-room promise that never lands as a commit is visibly nothing, and no process exists to be re-triggered into repeating it

OCG failure class	Disposition here	Why
Speculative prose before cheap probes	cultural, not enforced	the references-for-claims habit; no gate
Over-reporting incremental progress	dissolved	progress has non-room surfaces (board, notifier, reports); a room drifting into a status feed is misuse, checked at retrospectives
Retry/duplicate processing	minor	pull-on-wake reads are idempotent; archives are message-id'd

What we deliberately do not build, each unnecessary at wake-based participation: admission governor (no unsolicited invocations to admit), response leases (turns are granted by wakes or by the chair, visibly), egress governor and semantic firewall (speech carries no authority; a human is present; nothing needs suppressing), the action-cascade router (every wake is already a deliberate act), incident manager (incidents never enter rooms), semantic dedup (no duplicate-generation pressure).

Standing contingency. If any “dissolved” class reappears in archived transcripts — retrospectives read them — the governor’s own shadow-first instrumentation with predeclared go/no-go gates is the named retrofit path. We adopt the playbook and hold the machinery.

6 MVP: Slack

Slack first, for capability reasons, not taste: Slack bots can see other bots’ messages with ordinary scopes, so a multi-seat room actually works with **native per-seat identity** — each seat its own app, name, and avatar — and cheap channels give each deliberation a home of its own. Free-tier retention limits don’t matter *by design*: the archived transcript in files is canonical, so the transport is disposable.

Shape:

- One workspace. **One channel per deliberation** (delib-YYYYMMDD-slug), created at open and archived at close; the channel list is the standing index of deliberations, active and past. The opening message — pinned — carries question, roster, synthesizer; the closing message carries the conclusion references.
- Each seat is its own Slack app with its own token; the human is himself.
- **Pull-on-wake:** a woken seat reads the channel history and posts through its own token. No event subscriptions, no listener process — nothing is resident even technically.
- Wakes are manual at MVP: the operator opens seat sessions exactly as he does today. **Live-span is the first mode** — seat sessions stay up for the deliberation’s duration, the natural shape while wakes are manual. Per-turn waking (wake, read, speak, sleep) is the second experiment, the one that matters once wakes automate; cost per turn is recorded whenever either runs.
- Close: the synthesizer commits conclusions in the workspace; the transcript archives under the owning project (deliberations/<date>-<slug>.md, a two-line header — question, roster, conclusion references — over the transcript).

- Build: a thin CLI — read-channel, post-as-seat — a few dozen lines against the Slack API. No ledger changes.

7 What the MVP validates — and what it cannot

End-state need	MVP verdict
Human in the loop, not middleman	Tested directly — the felt-friction question the design exists to answer
Two independent seat contexts beat one session role-playing both	Tested by the pilot. Falsifier, recorded honestly: if the second seat contributes nothing the first couldn't have produced alone, the premise of cross-seat deliberation is weakened
Conclusions land; rooms don't become where decisions die	Tested per deliberation — synthesis committed, yes or no; the landing-rate metric starts at $n=1$
Wake/turn economics	Measured — live-span first; per-turn when it runs; the comparison decides how automated wakes should be shaped
Latency tolerance of turn-grain deliberation	Partial — manual wakes are human-paced; validates session-grain deliberation only, says nothing about automated sub-day loops
Wake-based participation needs no governor machinery <i>at scale</i>	Not validated. Nothing is resident in the MVP by construction, so the claim is untested until wakes automate; the check is retrospectives reading archives against the table in Section 5, with the contingency standing
Identity/attribution clarity	Mostly tested on Slack (native identities); the Telegram adaptation would leave it open (Section 8)
Transport disposability	Tested by construction: the Telegram adaptation (Section 8), or any future move, must cost one adapter against the same seat interface and archive format — if it costs more, the MVP was built wrong
Chat as the operating surface (approvals, reviews, launches through rooms)	Not tested — that end state waits on per-seat credentials; the MVP only proves the deliberation tenant of it
Multi-human rooms, privacy boundaries	Not tested

8 Adapting to Telegram (and what it costs)

If a phone-native Telegram room is wanted — it is where this hive's notification stream already lives — one platform rule reshapes the design: **Telegram bots cannot see messages from other bots**, a deliberate anti-loop measure (core.telegram.org/bots/features) that the OpenClaw stack has also encountered ([openclaw#408](#)). One bot per seat therefore cannot work. The adaptation is a single *bridge bot*: every seat posts through it with an identity prefix ([`omegahive-dp`]), it sees human messages normally, and it holds the canonical message-id'd log — Telegram becomes the human's window onto a log the bridge owns. Costs, stated plainly: no native seat identity (attribution rests on prefixes — the identity-confusion class reopens), no cheap per-deliberation

rooms (deliberations serialize through one group, or groups are made by hand), and every seat's traffic flows through one token (the bridge becomes a small piece of infrastructure the Slack design simply doesn't need). Workable, degraded — which is why Slack comes first and Telegram is the adaptation, not the reverse.